

ENTREGA FINAL DE PROYECTO

Riftbound Rules Chatbot

Juez de mesa con RAG, visión de cartas y verificación de rulings

Facundo Nahuel Godoy · proyecto individual

Links de acceso (el docente evalúa acá)

El producto es el repositorio y la app publicada. Este PDF es el índice.

Recurso	URL
Repositorio GitHub	https://github.com/FacuNGodoy/riftbound-rules-chatbot
Aplicación en funcionamiento	https://riftbound-rules-chatbot.onrender.com
Video de demo	https://youtu.be/I7B1EFWfUKE
Prototipo Figma (3 pantallas)	https://www.figma.com/make/AGMq8C6qLXKILCSnCtWujt/Rifbound---Bienvenida--Copy-?t=0m2XhVKWIFPBIIUx-1
Log de sesión real	eval/sesion_real.md en el repo

La app está publicada en Render. El primer acceso después de un rato inactivo puede tardar ~50 s (plan gratuito). Abrir la URL antes de la corrección.

Cómo verificar en 2 minutos: abrir el repo (README + commits), abrir la URL pública, preguntar “Si counterean un spell con Repeat, ¿se resuelve el Repeat?” y chequear que cite 820. Una pregunta de combate (Deathknell vs heal) debería mencionar la regla 320: durante Cleanup no se resuelve la chain.

PARTE 1 — El proyecto como aplicación real

Sección 1 · Equipo y proyecto

Integrantes

Nombre	Rol en el desarrollo
Facundo Nahuel Godoy	Producto, backend, prompts, ingestión del reglamento, base de cartas, eval de juez, UI y publicación

Nombre del proyecto: Riftbound Rules Chatbot. Es el mismo problema de la entrega de medio ciclo, ahora implementado: un jugador en partida no puede hojear el Core Rules para decidir si un Deathknell se resuelve antes o después del heal de Combat Cleanup. Discord tarda y a veces inventa. La app responde en español, cita reglas y texto de carta, y se abstiene si no tiene evidencia.

Público objetivo: jugadores casuales y competitivos de Riftbound (PC, durante o después de la partida) y jueces amateur que necesitan citar el texto, no la intuición.

Sección 2 · Arquitectura técnica

Entrada: texto y, opcionalmente, fotos. El frontend manda FormData a /ask-stream. FastAPI orquesta. Salida: veredicto, confianza, explicación y citas. IA: Gemini Vision en producción (minicpm-v local), Gemini draft y verifier. Tradicional: búsqueda de cartas en JSON, Chroma, pineado de mecánicas, atajos determinísticos, validación de IDs de cita. Memoria persistente de conocimiento: ChromaDB en disco (reglamento indexado). Memoria de sesión: conversation_history en RAM (últimos 10 turnos), se limpia al recargar o POST /reset. No hay base de usuarios.

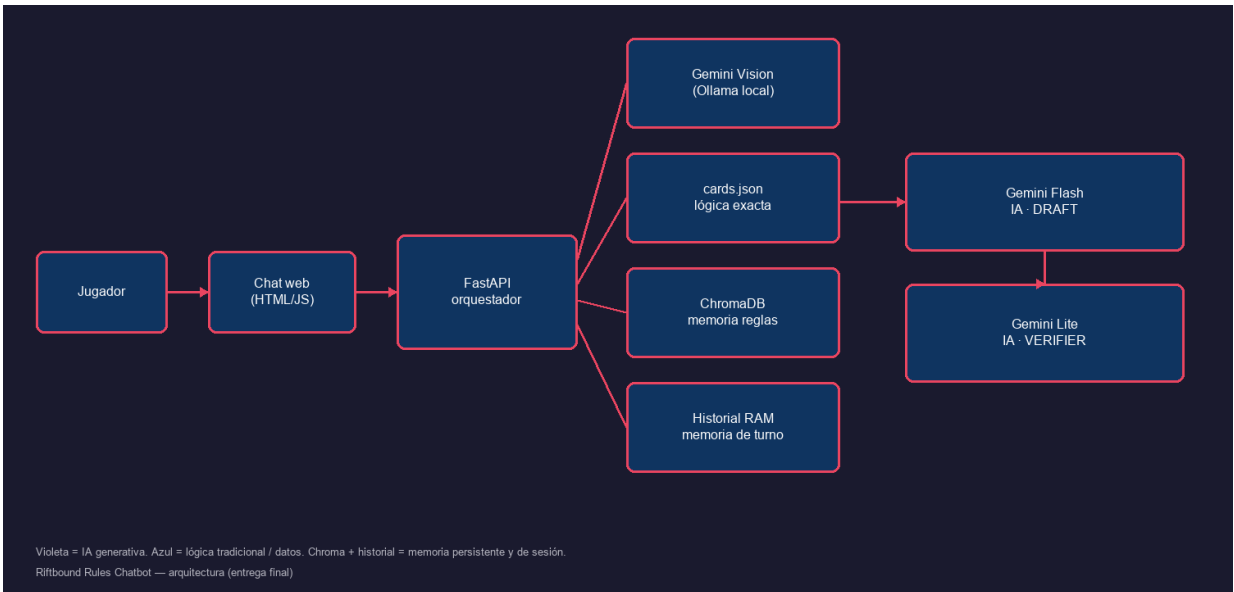


Figura 1. Arquitectura general. Componentes IA vs lógica tradicional.

Orquestación multi-rol (no es un framework tipo LangChain): el código propio decide el ciclo. Retrieval elige chunks y cartas. Draft redacta. Verifier ataca el borrador y exige JSON con citas. Si el veredicto es NO_RESUELTO y trae missing_rules_query, hay una segunda pasada de retrieval. finalize_ruling() tira abajo citas que no estén en la evidencia.

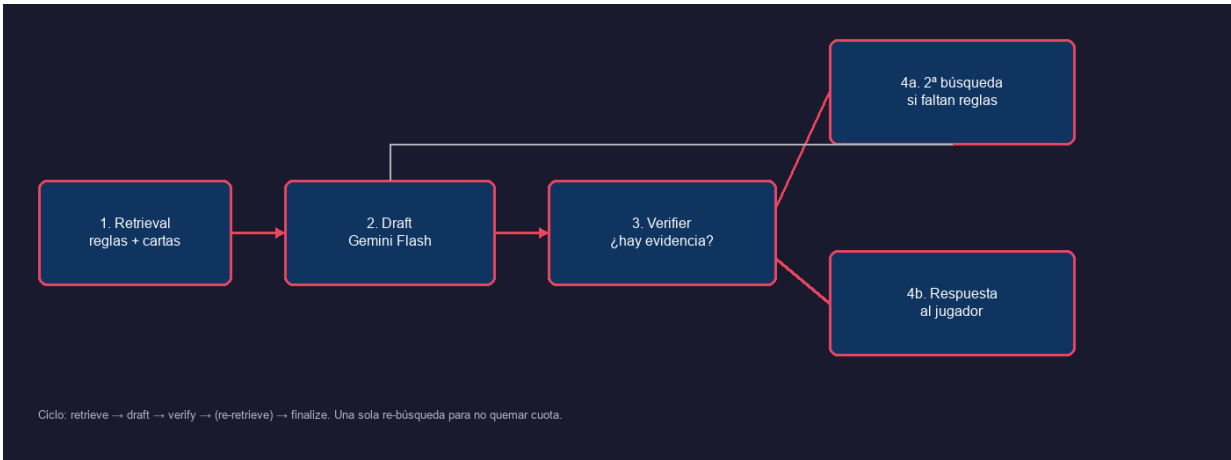


Figura 2. Flujo de agentes / ciclo de decisión.

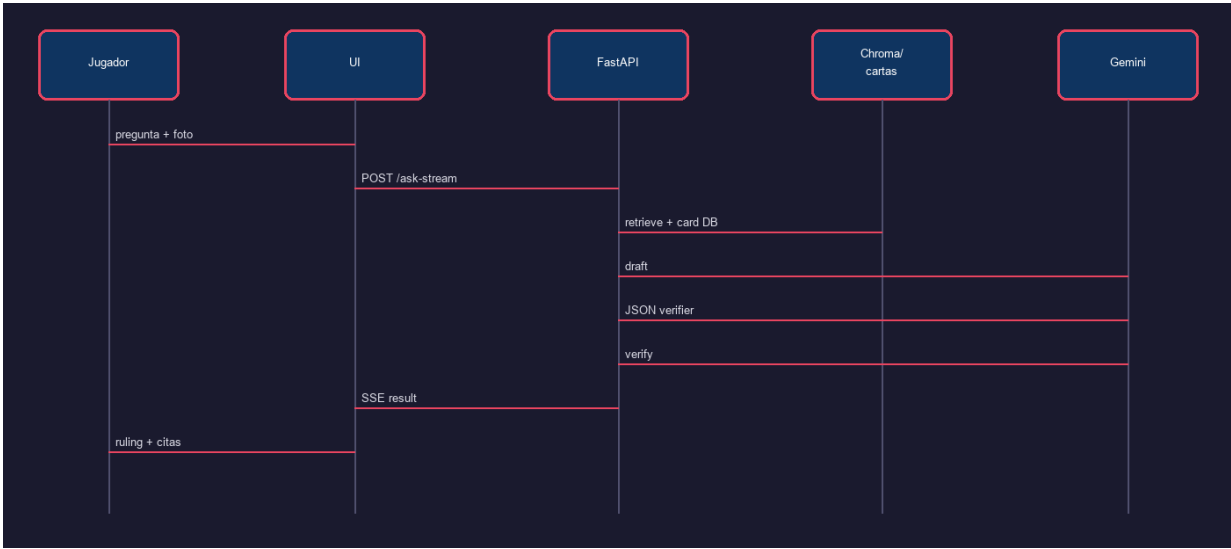


Figura 3. Diagrama de secuencia de una consulta completa.

Evolución respecto de la entrega de medio ciclo: el diseño ya planteaba RAG + visión + Gemini. Lo que se implementó después es el ciclo de decisión (draft → verifier → re-retrieve), la validación determinística de citas, atajos para interacciones ya juzgadas, el streaming SSE para no dejar la UI en silencio, y la publicación en Render. La memoria persistente no es un chat histórico en la nube: es el índice Chroma del reglamento, que sobrevive al reinicio. El historial de la partida vive en RAM a propósito (privacidad).

Código Mermaid equivalente (anexo de arquitectura):

```
flowchart LR
    U[Jugador] --> UI[UI Chat web]
    UI --> API[FastAPI]
    API --> V[Gemini Vision / minicpm-v local]
    V --> C[C[cards.json]]
    C --> R[R[ChromaDB]]
    R --> H[H[historial]]
    H --> D[D[draft]]
    D --> F[F[verifier]]
    F --> UI
```

Casos de uso (quién hace qué):

Actor	Caso de uso	Resultado
Jugador	Preguntar una regla o interacción	Ruling + números de regla
Jugador	Adjuntar foto de carta	Texto oficial desde cards.json
Jugador	Desambiguar (varias prints)	El bot pide el subtítulo
Autor / eval	Correr judge_cases.json	Log de aciertos/citas

Sección 3 · Stack tecnológico

Componente	Tecnología	Por qué esta y no otra
Frontend	HTML + JS + CSS	Una pantalla de chat. React no sumaba al TP y complicaba el deploy en PC propia.
Backend	Python / FastAPI	El ingest y Chroma ya eran Python. FastAPI sirve estáticos y SSE sin un Node extra.
Base de datos	ChromaDB + JSON	Chroma para RAG del reglamento, con el índice ya construido y versionado. SQLite/Postgres no aportan: no hay usuarios. Las cartas van en JSON para no alucinar el texto.
Embeddings	API de Gemini (gemini-embedding-001)	Empezó con sentence-transformers local, pero eso arrastra PyTorch y el plan gratuito de Render (512 MB) moría por falta de memoria. Los embeddings por API son multilingües y dejan el servidor sin modelos cargados. Medido: la suite de juez dio el mismo 5/7 antes y después del cambio.
Modelo de IA	Gemini Flash + Gemini Vision; minicpm-v local	Qwen3 8B local no obedecía el prompt de juez. Flash vigente: gratis y breve. Gemini solo identifica número/nombre de la foto; cards.json aporta el texto exacto. En local puede usarse minicpm-v.
Orquestación	Código propio (draft/verify/retry)	LangChain hubiera tapado el ciclo que hay que explicar. n8n no corre bien el retrieval de 960 cartas + reglas pineadas.
Despliegue	Render (Docker, plan Free)	Da una URL HTTPS estable sin depender de la PC. El límite de 512 MB obligó a sacar PyTorch: el índice Chroma se construye antes y los embeddings/visión salen por Gemini API.

Sección 4 • Evidencia de funcionamiento

Capturas de la aplicación publicada en Render (<https://riftbound-rules-chatbot.onrender.com>), no de un mock. El flujo: el jugador entra, pregunta o adjunta una carta, y obtiene veredicto, confianza, explicación y evidencia clickable.

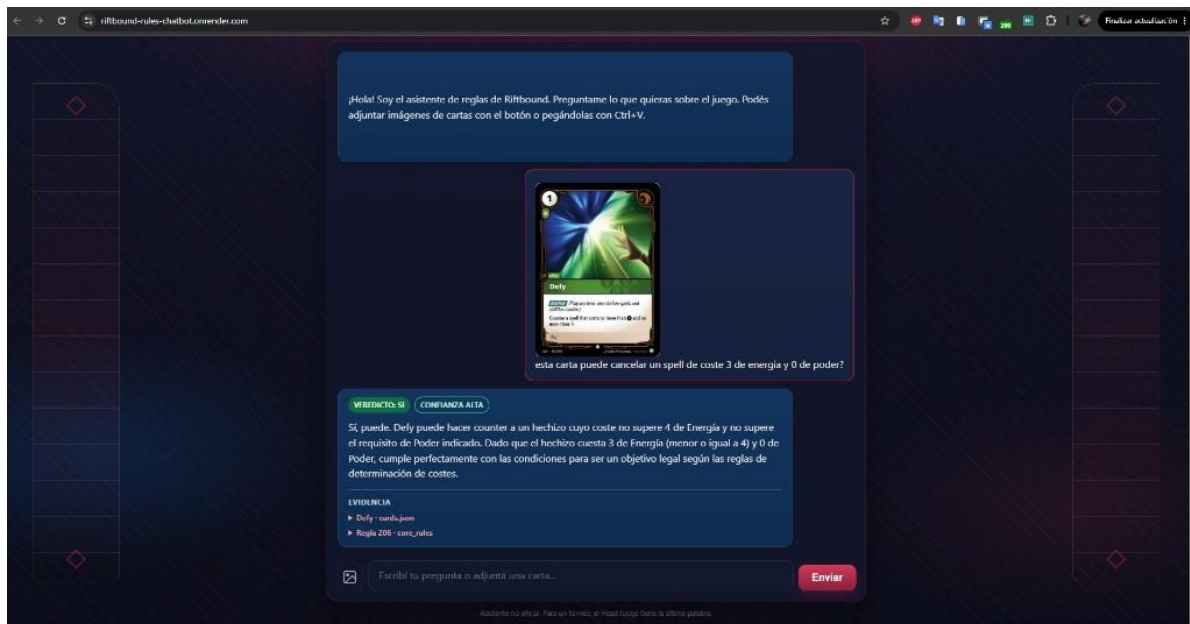


Figura 4. Producción: foto de Defy identificada por Gemini Vision. Veredicto SI, confianza ALTA, evidencia Defy (cards.json) y regla 206.

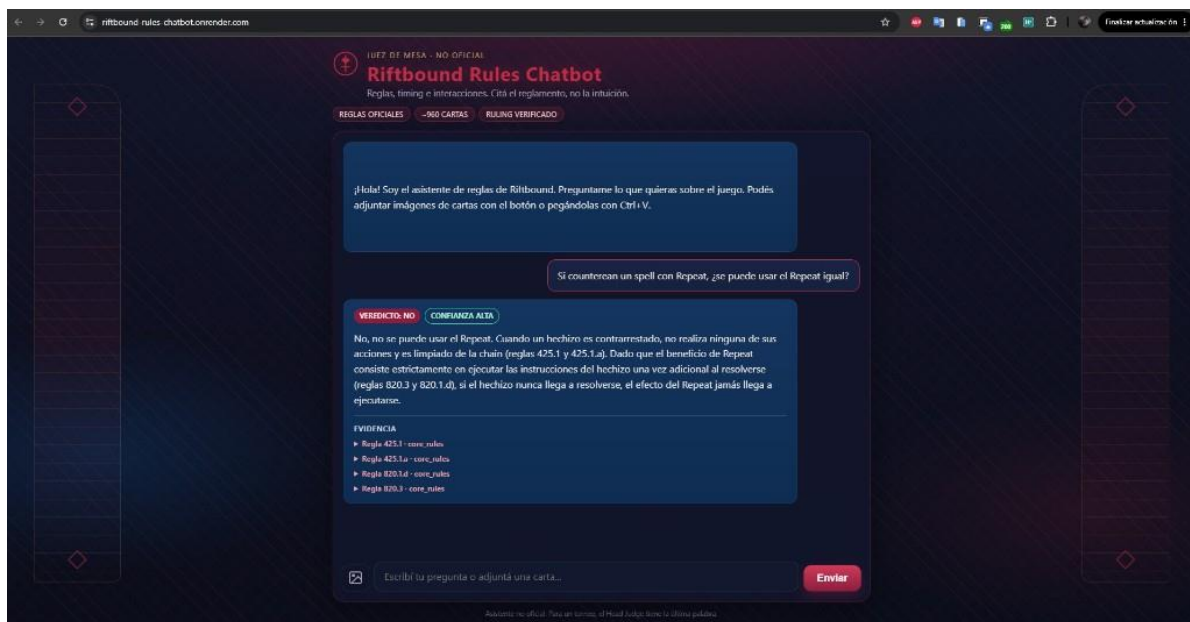


Figura 5. Producción: Repeat + Defy. Veredicto NO, reglas 425.1 y 820.1: si el hechizo es countereado no hay resolución y Repeat no se ejecuta.

Las tres pantallas de diseño (bienvenida, chat, config) siguen en Figma; abajo se conservan como prototipo de interfaz, no como el sitio live.

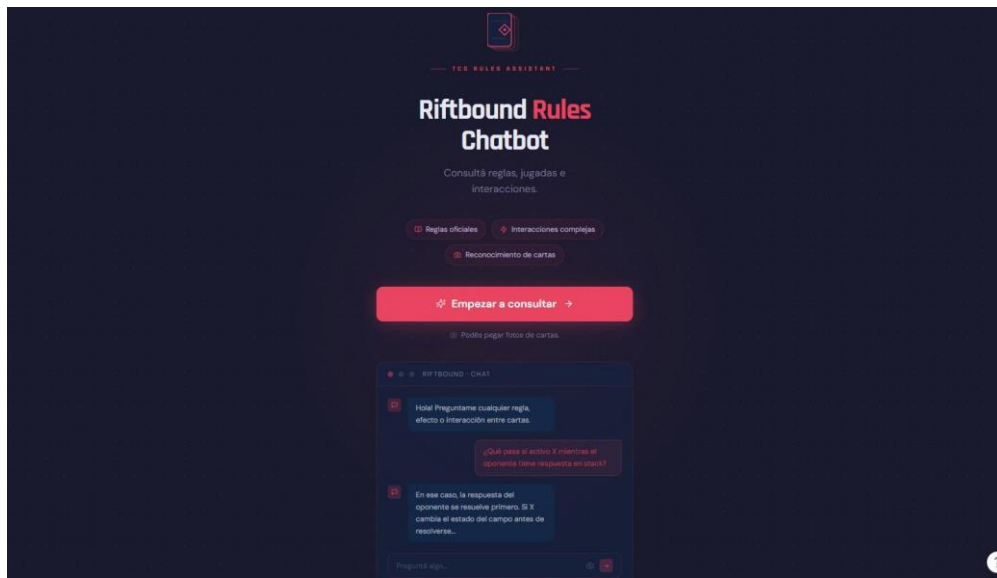


Figura 6. Home / bienvenida (prototipo Figma).

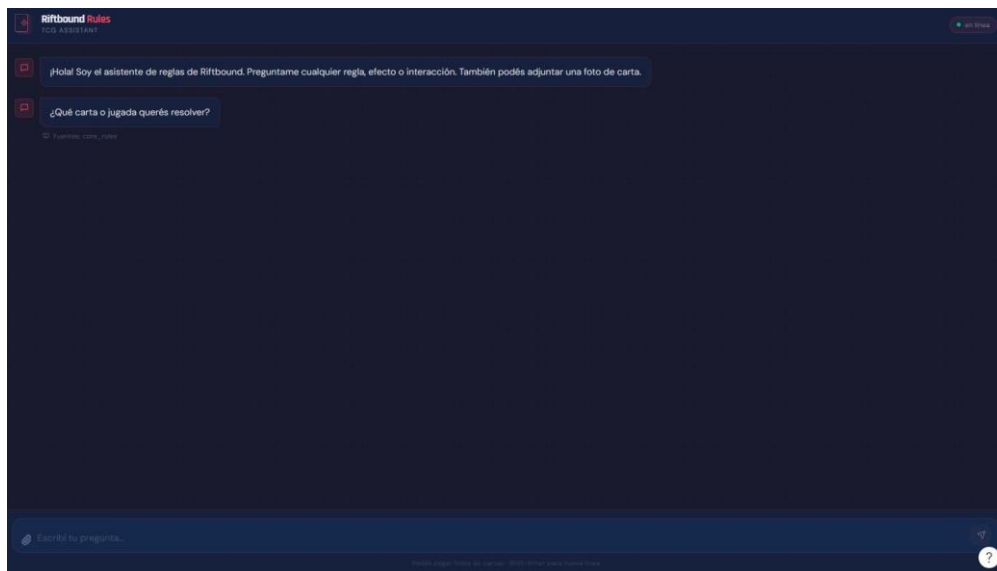


Figura 7. Flujo principal de consulta (prototipo Figma).

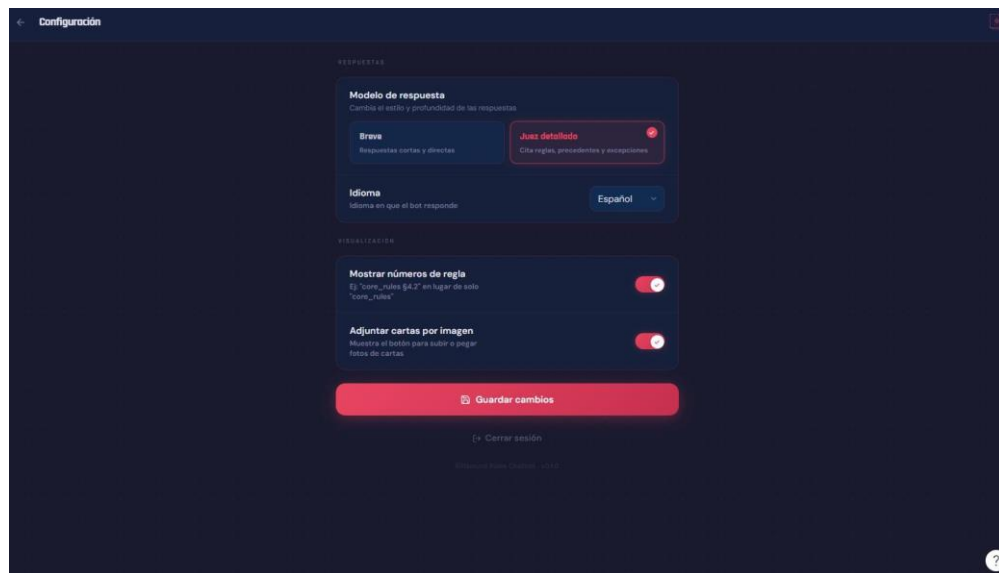


Figura 8. Configuración objetivo (Figma). En producción el modo es siempre juez con citas.

Output de la IA visible: cada respuesta trae veredicto (SI/NO/RESUELTO/...), confianza, explicación en español y chips de citas. El frontend muestra estados intermedios (buscando, generando, verificando) vía SSE.

Log de sesión real (datos de juego, no lorem): 2026-08-27, 8 casos de juez. Pipeline LLM 5/7 (71,4 %). Ejemplo — pregunta: «Si counterean (Defy) un spell con Repeat, ¿se puede usar el Repeat igual?». Veredicto NO, regla 820.1: Repeat se ejecuta al resolver; si hay counter, no hay resolución. Detalle completo en eval/sesion_real.md y eval/results_run1_backup.json.

Sección 5 · Evaluación UX/UI

5.1 Heurísticas de Nielsen

Heurística	¿Cumple?	Evidencia
Visibilidad del estado	Sí	Spinner + textos de status del server (searching / draft / verify). El jugador ve que no se colgó.
Coincidencia con el mundo real	Sí	Vocabulario TCG (countereado, chain, Might), no traducciones inventadas. Navy/magenta de herramienta de juego.
Control y libertad	Parcial	Hay /reset al recargar. No hay undo de un ruling ya dicho: se hace otra pregunta. Aceptable para un juez, no para un editor.
Consistencia y estándares	Sí	Chat clásico: burbujas, adjuntar, Enter para enviar. Citas como metadatos, no como otro producto.
Prevención de errores	Parcial	Si la carta es ambigua (varias prints) el bot pide el subtítulo en vez de rulinear la equivocada. No hay confirmación antes de gastar cuota.
Reconocimiento sobre recuerdo	Sí	Placeholder, chips de citas, preview de imagen. No hay que recordar números de regla para preguntar.
Flexibilidad y eficiencia	Parcial	Ctrl+V de fotos, números OGN-xxx y chips de ejemplo (Defy, Deathknell, Repeat).
Estético y minimalista	Sí	Una columna, sin arte de campeones (copyright y distracción). Config no está en el prototipo código para no competir con el chat.
Ayuda a reconocer errores	Sí	Si Gemini se queda sin cuota o falla la visión, el mensaje dice qué pasó (cuota vs Ollama caído), no un 500 vacío.
Ayuda y documentación	Parcial	El saludo inicial explica foto + pregunta. El README es para quien clona, no un help in-app largo.

5.2 Público objetivo

El usuario final no es un data scientist: es alguien con el mazo en la mesa. Por eso hay un solo campo de texto, español, y la respuesta va al veredicto antes que al ensayo. El lenguaje visual (contraste alto, sans-serif) se pensó para leerlo con mala luz. Prueba informal: se usaron preguntas reales de discordia de partida (Rex vs Yi / Deathknell, Hidden Blade + Zhonya, Kennen desde Hidden). Feedback: sirve cuando cita el número de regla; no sirve si se pone verboso o si cambia el timing. Eso empujó el verifier y los pineados de la 320. No hubo ronda formal de usability testing con grabación.

Sección 6 · Ciberseguridad

Riesgo	Tipo	Medida o decisión
Inyección de prompt	Prompt injection / OWASP LLM01	El usuario no edita el system prompt. La query viaja como pregunta. El verifier solo puede citar IDs que ya están en la evidencia recuperada.
Exposición de API keys	Secretos en código	GEMINI_API_KEY en .env, listado en .gitignore. .env.example sin secreto. La clave que estuvo hardcodeada en una versión local se saca del repo; conviene rotarla en Google AI Studio.
Fotos y preguntas del jugador	Privacidad	No se guardan imágenes ni historial en disco. En producción la foto se envía a Gemini solo para identificar la carta; el historial vive en RAM. En local puede usarse Ollama para que la imagen no salga de la PC.
Acceso no autorizado a la URL pública	Autenticación / superficie	No hay login: es un juez público de reglas (dato no sensible). Render expone solo FastAPI, no el filesystem. Riesgo residual: abuso de cuota Gemini si alguien bombardea /ask.
Alucinación presentada como regla oficial	Integridad / supply de la respuesta	Abstención si no hay citas válidas. cards.json es la fuente del texto de carta, no el LLM. Atajos determinísticos en interacciones ya validadas.

Sección 7 · IAs usadas en el co-work

Herramienta	Para qué	Aportó bien / mal / sorprendió
Claude / Cursor	Backend, prompts, eval, este informe	Bien para iterar retrieval y el ciclo draft/verify. Mal cuando inventaba timing; hubo que pinear la 320 a mano.
Gemini Flash	Modelo de producción del juez	Bien en preguntas simples y en español breve. Mal en Cleanup/Deathknell hasta meter verifier + reglas críticas.
Qwen3 8B (Ollama)	Intento de juez 100 % local	Mal: verboso, “contado” por countereado, no nombraba la carta. Descartado.
Gemini Vision / minicpm-v	Leer número/nombre de la foto	Gemini se usa en Render y minicpm-v localmente. Ambos solo identifican; cards.json es la fuente del texto. Al principio minicpm-v mezclaba prints y se agregó Name + Subtitle.
Figma Make / Leonardo	UI de 3 pantallas y paleta	Figma = entregable. Leonardo = clima visual, no el producto. Make pisaba pantallas al iterar.

Reflexión: sin co-work con IA no existiría el cruce de ~960 cartas + Core Rules en un cuatrimestre de una sola persona (scraper, chunking, system prompt de juez, suite de casos). La parte que la IA hizo mal y hubo que corregir es el timing de combate: los modelos afirman con seguridad que Deathknell se resuelve “de inmediato”. Eso no se arregló con más prosa en el prompt; se arregló con evidencia forzada, un segundo agente que tiene que atacar el borrador, y un atajo de código en Hidden Blade + Zhonya. El criterio humano fue filtrar modelos, no copiar la primera arquitectura.

PARTE 2 — IA local en el proyecto

1. Papel de un LLM/SLM local. Ya hay un SLM local disponible: minicpm-v, subagente de visión. En Render ese rol lo toma Gemini porque no hay GPU, pero localmente evita enviar fotos a terceros. No reemplaza a Gemini como juez. Se probó reemplazar el agente principal con Qwen3 8B y no cumplió el contrato de juez (brevedad, vocabulario, citar la carta). Un LLM local más capaz (Llama 3.1 70B, etc.) reemplazaría el draft/verifier para no mandar jugadas de un grupo a Google, pero no entra en una PC de escritorio razonable. El rol realista hoy: visión + quizá un filtro de prompt injection, no el Head Judge.
2. Aporte al usuario. La visión cambia lo que puede pedir: saca la foto de la mesa y no tiene que tipear OGN-195. La variante local es más privada (la imagen no viaja a Gemini) y no tiene costo por token de visión; la variante publicada permite la misma UX sin GPU. No es más rápido en el ruling: el cuello es Flash + verifier. Un juez 100 % local mejoraría privacidad y seguiría andando sin internet; hoy, si se corta la API, el bot se abstiene.
3. Aporte profesional. Tener Ollama en el loop permitió comparar calidad real, no slides: se vio el vocabulario roto y se documentó. Si el bot se usara en un club o tienda, un modelo local dejaría logs de preguntas (qué interacciones duelen) sin subir partidas a un proveedor. Eso cambia el trabajo día a día: eval sobre casos propios, no sobre la memoria del modelo de turno que Google retire (ya pasó con gemini-2.0-flash).
4. Limitaciones vs API en la nube. Hardware: esta PC corre minicpm-v; no corre un juez al nivel de Flash. Calidad: Qwen 8B falló el caso de uso. Cuota y retiro de modelos: el problema de la nube. Mantenimiento: hay que pullear pesos, versionar Modelfile y re-correr judge_cases.json cada vez que Ollama actualiza. Conclusión: híbrido a propósito (visión local opcional y visión/juez por API en producción), no por no haber pensado lo local.

Entregable opcional (Ollama): en una terminal, con Ollama instalado, `ollama run minicpm-v` o `ollama run llama3.2` y preguntar p.ej. “En Riftbound, ¿Deathknell se resuelve antes o después del heal de Combat Cleanup?”. Pegar captura en el anexo. Sirve para mostrar que lo local responde, y para contrastar con la respuesta verificada de la app (regla 320).

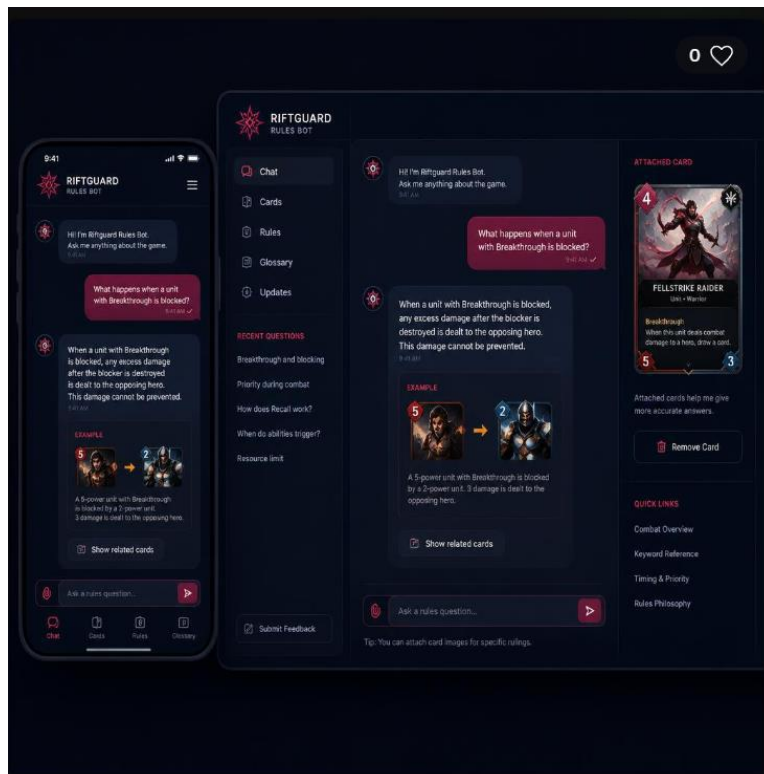


Figura 9. Exploración de estilo (Leonardo.ai) de la entrega anterior: paleta, no el producto.

Anexo A · Cómo reproducir

- Clonar el repo, pip install -r requirements.txt, copiar .env.example → .env.
- python ingest.py && uvicorn app:app --port 8000
- Túnel: cloudflared tunnel --url http://127.0.0.1:8000
- Eval: python eval/run_eval.py --base-url http://127.0.0.1:8000

Anexo B · Extracto del log de sesión (2026-08-27)

Pregunta: Si juego de Hidden a Kennen, Keeper of Balance, ¿puede stunear cualquier battlefield? → NO / ALTA / 811.1.d.2: los targets del play effect desde Hidden tienen que estar en ese battlefield.

Pregunta: Si counterean (Defy) un spell con Repeat, ¿se puede usar el Repeat igual? → NO / ALTA / 820.1: Repeat ocurre al resolver; sin resolución no hay Repeat.

Pregunta: Si Nidalee, Cat Form muere en combate pero quedan unidades, ¿roba igual? → NO / ALTA. “I remain after combat” refiere a la carta, no al jugador (053).

Pregunta (Rex vs Yi, Deathknell): ¿el daño se resuelve antes o después del heal? → INFORMATIVO / ALTA / 320: triggerea en Cleanup y se resuelve después del heal. Yi sobrevive.

Pregunta: Hidden Blade vs Zhonya, ¿roba 2? → SI (código determinístico, 359.3.e.14). Reemplazo del kill ≠ mistarget. JSON completo: eval/results_run1_backup.json.

Fin del informe. Historia de trabajo: commits en GitHub (no un ZIP único). La app debe estar con el túnel levantado al momento de la corrección.